Proposed policy: scope-aware heuristics and selective, budget-aware review

Full design proposal, revised **20 October 2026**. A bounded literal-rule/state/allocation/stub core is implemented; semantic scope extraction remains pending. The gated execution runner is connected; live model compatibility remains unverified. See implemented behavior and limits. No empirical performance is claimed. This is the internal architecture of the approval policy, not a team workflow or a generic agent harness. It supersedes the high-level policy sketch in the earlier architecture supplement. Numerical examples are illustrative development candidates.

1. What this policy computes

Given the user's authorized task, the exact proposed tool call, relevant observations and remaining review budget, return **APPROVE**, **DENY**, or **ESCALATE**, with evidence and a reason.

The proposal has two distinct functions:

1. **Scope-aware heuristics:** determine whether the operation, resource, recipient, data and side effects fit the user's request. Resolve cases supported by explicit checks cheaply.
2. **Selective, budget-aware review:** for unresolved calls, determine whether a reviewer can resolve the uncertainty and whether that opportunity merits one of the remaining review slots.

The objective is to test improved task completion under attack and finite review budgets. Under the user's clarification, use official AgentDojo outcomes plus executed-call audit trails, as defined in the evaluation contract. This benchmark operationalization is not a universal no-unauthorized-call certificate. Budget scarcity changes review selection; it never supplies missing authorization.

1.1 Accepted scope: all standard suites

The user's latest correction supersedes the earlier banking-only direction. Design for **workspace, travel, banking and Slack**. Use a shared scope ledger/check engine/allocator with suite-specific tool semantics and effect normalization. The complete domain matrix, counts and evaluation implications are in Multi-domain policy scope.

The loaded v1.2.2 suites contain 97 user tasks and 74 per-suite tool registrations. Metadata covers all 74 registrations; the implemented shared core uses suite-specific validators and bounded literal scope construction. Examples involving a bill below remain illustrations, not restrictions on supported domains. Scope/check state and R4 assessment requests must carry suite identity. Keep semantics keyed by suite and registered signature rather than global tool name alone.

2. Internal architecture

```
flowchart TD
    U["User request and trusted follow-up instructions"] --> S["1. Scope
ledger: operations, targets, recipients, data and constraints"]
    O["Relevant observations with source IDs"] --> F["Fact store: evidence
```

```
only; no new permission"]
    C["Exact proposed tool call"] --> I["2. Action interpreter: semantics,
targets, data flows and side effects"]
    M["Verified per-suite tool semantics"] --> I
    S --> H["3. Scope heuristics: compare action with authorized scope"]
    F --> H
    I --> H
    H --> D{"Scope-check result"}
    D -->|Proven violation or invalid call| NO["DENY with evidence and
reason"]
    D -->|Required checks all satisfied| YES["APPROVE exact call"]
    D -->|Unresolved| A["4. Bounded semantic assessment of explicit
unresolved checks"]
    A --> V["Validate output and recompute scope-check vector"]
    V --> Q{"Updated scope result"}
    Q -->|Required checks satisfied| YES
    Q -->|Violation| NO
    Q -->|Still unresolved| R["Reviewability filter: question, evidence and
why review adds value"]
    R -->|Missing authority or inaccessible evidence| NO
    R -->|Reviewable| P["5. Review allocator: priority from necessity,
resolution value and consequence"]
    B["Read-only remaining review budget b"] --> T["Budget-dependent
threshold"]
    T --> SELECT{"Priority reaches threshold and b is positive?"}
    P --> SELECT
    SELECT -->|Yes| E["ESCALATE with a targeted review question"]
    SELECT -->|No| NO
```

Outside this policy: the execution gate enforces the returned decision, reserves review slots, contacts the reviewer, and executes only an approved call. The policy does not execute actions, modify the authoritative budget, or use evaluator labels.

3. Component 1: an authorization-scope ledger

Representation

Use a task-local record with trusted instruction references and separately referenced observed facts. Example: a user asks to pay a named bill.

Field	Banking example
Requested goal	Pay the named bill
Authorized operations	Read that bill, resolve payment details, make the requested payment
Resource constraints	Named bill and relevant account/history observations
Recipient constraints	Explicit beneficiary or task-authorized resolution from observed bill/history

Field	Banking example
Amount constraints	Explicit amount or authorized calculation from observed task data
Date/recurrence	Requested payment timing and one-time/recurring scope; leave unstated details unresolved
Content constraints	Payment subject contains only task-relevant information
Side-effect constraints	No unrelated transfers, profile/security changes or duplicate payment
Delegation	Record whether the user authorized following a named document and the task limits of that delegation
Unresolved constraints	Missing/ambiguous recipient, amount, target transaction or instruction interpretation
Provenance	Original user instruction and separate observation IDs

A document can resolve details or delegated instructions within the task. It cannot automatically authorize unrelated action. If the same compromised document is the only evidence for a value, mark uncertainty honestly; a provenance tag cannot recover the hidden true value.

Construction and updates

- Begin with explicit user constraints and a small, documented supporting-action vocabulary for the selected suite. A supporting action must be needed for this task and remain within its resource/data limits.
- Treat any LLM-extracted ledger as a candidate interpretation. Check explicit addresses, dates and IDs against their original instruction spans. Retain unknowns rather than manufacturing a complete permission set.
- Permit narrow discovery when the user has authorized identifying a target. Store discovered identity as a fact linked to the original authorization; do not treat every retrieved ID as authorized.
- A user may explicitly delegate following a document (banking UserTask12 is an example). Record that delegation without treating arbitrary embedded claims as a new trusted user message. Broader interpretations remain uncertain; a reviewer approving one exact call does not rewrite general scope.
- Keep prior approvals as exact-call events. They are not blanket permission for later calls with different payloads or targets.

Justification: tool-name allowlists cannot distinguish a requested read from an unrelated private read. The ledger makes the boundary explicit and auditable. Its main risk is incorrect task interpretation; unresolved interpretation must remain visible downstream.

4. Component 2: interpret the actual action

Normalize each proposed call using its suite-specific trusted tool definition into:

`operation, resources, recipients/destinations, data_to_read, data_to_disclose, mutations, reversibility, argument_sources`.

Examples: `read_file` accesses one simulated file; `send_money` changes transaction state and can disclose information through its subject; `update_scheduled_transaction` modifies the fields its implementation actually applies. The inspected amendment tool conditionally applies fields using truthiness, so a proposed false recurrence or zero amount may not change state. Treat this as a benchmark semantic limitation to document, not something to silently patch during evaluation. Unknown tools do not inherit a harmless classification from their names.

Justification: the same tool can serve legitimate or unauthorized purposes. Normalized semantics let one recipient/content-scope rule cover several tools. Incorrect metadata remains a failure mode, so annotate source locations and version it with the benchmark.

5. Component 3: scope-checking heuristics

For each action, produce a vector of required checks. Each check is **SATISFIED**, **VIOLATED**, **UNRESOLVED**, or **NOT_APPLICABLE**, with evidence references and a reason.

Check	Question	Example violation versus uncertainty
Operation	Is this operation requested or a justified supporting step?	Deletion during a lookup-only task: violation
Resource	Is this the authorized record/file/time range?	Wrong known scheduled transaction: violation; two matching records: unresolved
Recipient	Does every destination match the authorized identity/scope?	Beneficiary contradicts explicit user instruction: violation; ambiguous bill beneficiary: unresolved
Content/data	Are reads and disclosures limited to authorized information?	Unrelated account history in payment subject: violation; unresolved subject content: unresolved
Side effects	Are all material effects within authorization?	Requested lookup but proposed transfer: violation
Provenance	Is the permission supported by trusted instructions?	Tool text alone instructing a new disclosure: cannot establish permission

Frozen rule precedence

1. Invalid schema/arguments → **DENY**, with a distinct operational reason.
2. Any required check proven **VIOLATED** → **DENY**. The semantic assessor cannot override an explicit constraint.
3. Every applicable required check **SATISFIED** → **APPROVE**, subject to the action-class evidence requirements below.
4. Otherwise → assess the unresolved checks; never assume they passed.

Narrow fast paths and action-class requirements

- **Low-consequence helpers:** approve an applicable account lookup or similarly verified helper when it enables the task and has no material out-of-scope effects.

- **Scoped reads:** require resource and requested data scope to be established. “Read-only” by itself is insufficient.
- **Writes/disclosures:** require the operation, target, payload scope and destinations to be established. Authorized writes may pass; not every write must consume a review slot.
- **Unknown semantics:** remain unresolved until trusted metadata establishes what execution would do. If that cannot be established, deny.

A recipient absent from the literal prompt is not a sufficient denial rule. Conversely, a recipient mentioned in an external document is not sufficient permission. Both cases require tracing the user-authorized lookup or operation.

Justification: four-state checks prevent a common mistake—treating absence of a detected violation as proof of authorization. Deterministic early decisions save model/review cost while keeping the approval standard explicit.

6. Component 4: resolve uncertainty, then test whether review is useful

Bounded semantic assessor

Invoke a fixed, structured LLM assessment only for unresolved checks needing interpretation, such as whether a payment instruction remains within the requested rent amendment. Provide the exact call, ledger constraints and bounded relevant evidence, with source trust marked.

Request: check ID, proposed status, supporting/conflicting evidence, unresolved question, consequence, and whether the reviewer has evidence that could settle it. Require no hidden reasoning transcript or free-form confidence probability. Parsing and citation checks do not prove the assessment correct; independently evaluate its errors on development cases.

After assessment, reapply the same decision table. If all required checks are supported, approve. If a violation is established, deny. A format failure or hallucinated evidence leaves the affected check unresolved.

Reviewability filter

Escalation requires all three:

1. A specific remaining authorization question.
2. Evidence the reviewer can actually inspect.
3. A plausible benefit from focused second assessment, rather than merely repeating an unsupported guess.

The reviewer may share some evidence with the assessor; independence is not guaranteed. Targeted attention or a more suitable review context is a hypothesis to test, not an assumption of superior judgment. Log why the assessor could not settle the question and what review is expected to add.

Uncertainty	Reviewable?	Why
Payment subject is available, but whether it discloses unrelated account history needs interpretation	Potentially yes	Reviewer can inspect exact content against the task

Uncertainty	Reviewable?	Why
Two historical payments match a vague refund request and no evidence distinguishes them	No	Reviewer cannot invent missing user intent
Tool semantics unknown and no trusted definition exists	No	Neither model can establish effects reliably
Assessor timed out, but complete call/scope/evidence exists	Potentially yes	Reviewer can perform the missing assessment

Justification: review is valuable only if it can affect a decision using legitimate evidence. A review slot should not be spent to obtain consent that the simulated reviewer cannot give.

7. Component 5: selective, budget-aware review allocation

Define review priority explicitly

For a reviewable unresolved call, use the following ordinal ratings, each with evidence:

Feature	Candidate levels	Interpretation
Necessity n_t	0, 0.5, 1	No established contribution; plausible task progress; concrete prerequisite/no evident scoped alternative
Resolution value v_t	0, 0.5, 1	No inspectable evidence; potentially resolving evidence; a concrete question with directly relevant evidence
Consequence c_t	0, 0.5, 1	Low-consequence operation; bounded mutation/disclosure; sensitive disclosure or substantial irreversible effect

These are rubric levels, **not probabilities**. “No evident alternative” means no alternative established from available context; it is not knowledge of all future plans. Missing evidence cannot receive the highest rating by default. If $n_t = 0$ or $v_t = 0$, deny rather than escalate.

An illustrative normalized priority is:

$$p_t = n_t v_t \left[\alpha + (1-\alpha) c_t\right], \qquad 0 < \alpha \leq 1.$$

Use $\alpha = 0.7$ as a **development starting point only**. Necessity and resolution value gate the score; consequence increases the priority among useful, reviewable candidates. This avoids ranking an irrelevant dangerous action above a necessary ambiguous step. Clear violations have already been denied and never reach this formula.

Price a review slot according to scarcity

For $b > 0$ remaining slots:

$$\tau(b) = \tau_0 + \frac{\delta}{b}, \qquad \delta \geq 0.$$

Review selection is:

$$\text{select_review}_t = \mathbf{1}\left[b > 0 \ \wedge \ n_t > 0 \ \wedge \ v_t > 0 \ \wedge \ p_t \geq \tau(b)\right].$$

Selected → **ESCALATE** with the exact question and evidence. Not selected → **DENY**, not approve. At $b = 0$, the gate enforces the mandatory exhausted-budget denial. The policy may preserve “would require review” in its diagnostic reason without contacting a reviewer.

Candidate values $\tau_0 = 0.4$, $\delta = 0.2$ illustrate the mechanism: the threshold is 0.45 at four slots, 0.50 at two, and 0.60 at one. A high-necessity, strongly reviewable low-consequence call scores 0.70 and remains eligible with one slot. A candidate with $n_t = 1$, $v_t = 0.5$, $c_t = 0.5$ scores 0.425 and is rejected even at four slots. These examples explain behavior; they do not justify the numerical choices empirically.

Justification: reducing available slots increases the opportunity cost of spending one now. This is a transparent reservation heuristic, not an optimal online allocator. It does not predict future calls or use evaluator labels. It can fail by reserving attention that is never used; measure that explicitly.

State and invariants

Store the ledger, relevant facts, check results and decision history. Read remaining budget from the gate; do not create a second mutable budget. Every actual reviewer request consumes a slot whether approved or denied; assessor calls consume API cost but not review slots. Do not let repeated paraphrases reset task budget or inherit a prior approval for a different call.

The complete policy returns decision, reason code, check vector, evidence IDs, assessor status, reviewability question, feature ratings, priority, threshold and observed remaining budget. Logs distinguish policy intention from the gate's effective action.

8. Worked banking trajectory (one domain example)

Illustration, not a saved benchmark result. For a named-bill payment task, the request may authorize looking up payment details rather than naming them literally.

Proposed action	Scope/evidence result	Policy behavior
Read the named bill	Directly relevant source	Approve if the required resource checks pass
Look up relevant history to interpret a payment	Potentially necessary supporting read	Check necessity and available tool granularity; avoid arbitrary record caps
Transfer an explicitly contradicted amount/recipient	Proven conflict with original user constraints	Deny without review
Add unrelated account details to the payment subject	Unrelated disclosure established by evidence	Deny; “small transfer” does not make disclosure harmless
Pay with an ambiguously interpreted bill field	Unresolved, with inspectable contextual evidence	Assess, then apply reviewability and priority if still unclear

For a reviewable unresolved candidate with $n_t = v_t = 1$, $c_t = 0.5$, the illustrative formula gives $p_t = 0.85$ and $\tau(2) = 0.50$: escalate. A reviewer may approve or deny that exact call; either consumes one

slot. If the uncertainty is a completely forged beneficiary in the sole compromised bill, review cannot be assumed to know the genuine beneficiary. Record this limitation and evaluate the actual benchmark outcome.

Never fill the ledger from hidden `_BILL_RECIPIENT`, `ground_truth()` or expected outputs. Those fields can explain offline scoring but cannot be runtime policy inputs.

9. Why these components, and what evidence would challenge them?

Design choice	Justification	Failure to look for	Controlled comparison
Scope ledger	Makes authorization dimensions explicit	Bad scope extraction blocks legitimate work or invents permission	Hand-audit development ledger/check errors
Scoped fast paths	Avoid spending review on established cases	Overbroad read/resource allowance	Same safety criteria with model-assessed clear cases; compare cost and errors
Explicit unresolved checks	Prevent unsupported default approval	Too many unknowns destroy completion	Track unresolved categories and resolution outcomes
Reviewability filter	Avoid review that cannot supply missing facts/authority	Filter rejects useful review	Small paired development reruns without the filter
Scarcity threshold	Reserve review for stronger opportunities	Necessary calls denied while budget stays unused	Same policy with a fairly tuned fixed threshold
Evidence-based priority	Explain which review opportunities are favored	Ordinal ratings are inconsistent or consequence dominates	Fixed rating protocol, error audit and sensitivity check

Required main comparisons remain full approve, full escalate and our developed policy. To support the budget-awareness claim, hold scope rules, assessor and reviewer fixed and compare budget-aware against fixed allocation with equal development tuning opportunity. Do not add all optional ablations if cost prevents the required comparison.

Evaluate AgentDojo-based TCR@k, attack success, attacked utility, clean utility, escalation totals with cohorts, component costs, failures with unused budget and exhaustion denials. Independently established unauthorized execution is an optional audit finding, not a required label for every episode. Reviewed-call success alone is confounded by selection: difficult calls are more likely to be reviewed.

10. What comes from prior work versus our proposal?

The following are **inspirations**, not evidence that this proposed architecture already works:

- Public Guardian policy separates risk, authorization and evidence, and discusses payload/destination scope. Borrow those distinctions, not its exact risk thresholds. Guardian template, policy.
- Gemini CLI demonstrates explicit rule representation and three decision outcomes. Borrow inspectable rule configuration. Policy engine.
- Anthropic describes early allowances and a staged LLM classifier. Borrow inexpensive resolution before costly assessment, without assuming read access is always within task scope. Auto-mode architecture.
- CaMeL motivates provenance/data-flow tracking. This ledger does not implement its capability interpreter or inherit its formal guarantees. Paper v2.

Our proposed contribution: combine explicit scope checks with a reviewability filter and an inspectable review-priority/scarcity rule under the handout's finite per-task budget. Neither the numerical rubric nor superiority over a fixed threshold is established by those references. They require development validation and held-out experiments.

11. Implementation order after agreeing contracts

1. Define scope ledger, action semantics and four-state check schema; review manually on diverse development calls.
2. Implement deterministic rules and precedence with an assessor stub.
3. Specify unresolved-check inputs/output with the semantic-assessment owner; verify failure handling.
4. Implement reviewability and feature rubric, initially logging scores before enabling reservation.
5. Integrate with the authoritative gate and reviewer; prove denied calls do not execute and budgets cannot overspend.
6. Pilot fixed and scarcity thresholds; tune only on development tasks, then freeze all ratings/configuration.
7. Run matched comparisons and report actual outcomes and failures.

Executed-call audit logging and official benchmark scoring are mandatory; independent authorization labeling is not a prerequisite under the user’s clarification. This initial policy needs no newly trained model. It is a rule-based architecture with optional LLM interpretation and a hand-designed budget allocation heuristic.